

Design patterns

Soon Phei, Tin

Topic Objective

- Coursework 1
- Introduction to design patterns
- Creation patterns
- Structural patterns

What is Design Pattern?

- **Design patterns** are typical solutions to commonly occurring problems in software design.
- They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.
- The pattern is not a specific piece of code that you can copy and paste, but a general concept for solving a particular problem.
- You can follow the pattern details and implement a solution that suits the realities of your own program.

What Can I Find In A Pattern?

- Most patterns are described very formally so people can reproduce them in many contexts.
- Here are the sections that are usually present in a pattern description:
 - **Intent** of the pattern briefly describes both the problem and the solution.
 - **Motivation** further explains the problem and the solution the pattern makes possible.
 - **Structure** of classes shows each part of the pattern and how they are related.
 - **Code example** in one of the popular programming languages makes it easier to grasp the idea behind the pattern.
 - The **consequences** are the results and trade-offs of applying the pattern.
 - etc

How Do You Use Design Pattern?

- Use it primarily as a communication tool instead of a design tool.
- Sometime it can become a cautionary tale – things to do or not to do.
- We don't jump on to a design pattern, rather we arrive at a design pattern.
 - When YOUR design come close to a design pattern, you should consider to lean toward that design pattern, rather than seeking for a design pattern in the very beginning.
 - Do not force yourself to use a pattern.

Benefits of Design Patterns

- Reusability: Design patterns provide solutions that can be reused in multiple projects, reducing the overall development time.
- Best Practices: They reflect the experience and insights of software development professionals, thereby representing best practices in the field.
- Understandability and Maintainability: They apply a consistent, well-understood, and well-documented approach to software design which makes the code easier to understand and maintain.
- Communication: The use of well-known design patterns can improve communication within the development team by providing a common vocabulary.

Drawbacks of Design Patterns

- Overuse: Sometimes, a simple solution is more efficient and easier to understand. Using a design pattern where it's not needed can lead to unnecessary complexity.
- Initial Overhead: Understanding and implementing design patterns can be time-consuming initially and may slow down the development process.
- Specificity: Design patterns are not one-size-fits-all solutions. They provide solutions for specific situations and might not be applicable to all cases.
- Dependence: Over-reliance on design patterns could limit the developer's ability to think outside the box and come up with innovative solutions.
- Difficulty in Learning: Some design patterns can be complex to learn and implement, especially for less experienced developers.

Classification of Design Patterns

- All patterns can be categorized by their *intent*, or purpose. This chapter covers three main groups of patterns:
 - **Creational patterns** provide object creation mechanisms that increase flexibility and reuse of existing code.
 - **Structural patterns** explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient.
 - **Behavioral patterns** take care of effective communication and the assignment of responsibilities between objects.

Creational Patterns

Creational design patterns are used to deal with object creation, aiming to create objects in a manner suitable to the situation.

- Flexibility: provide a way to make a system independent of how its objects are created, composed, and represented.
- Reusability: By defining separate factory classes or methods for object creation, the same code can be reused to get instances of classes rather than creating them again and again, making for less redundant code.
- Abstraction: Creational design patterns abstract the instantiation process. They hide the implementation details of the objects that are being created.
- Control Over Object Creation: Creational patterns give more control over the object creation process and encapsulate this process in a separate function or class, allowing the developer to change or refine what classes do without affecting other parts of the code.

5 creational patterns

- **The Factory Pattern** provides a simple decision making class that returns one of several possible subclasses of an abstract base class depending on the data that are provided.
- **The Abstract Factory Pattern** provides an interface to create and return one of several families of related objects.
- **The Builder Pattern** as name implies, is an alternative way to construct complex objects. This should be used only when you want to build different immutable objects using same object building process.

5 creational patterns

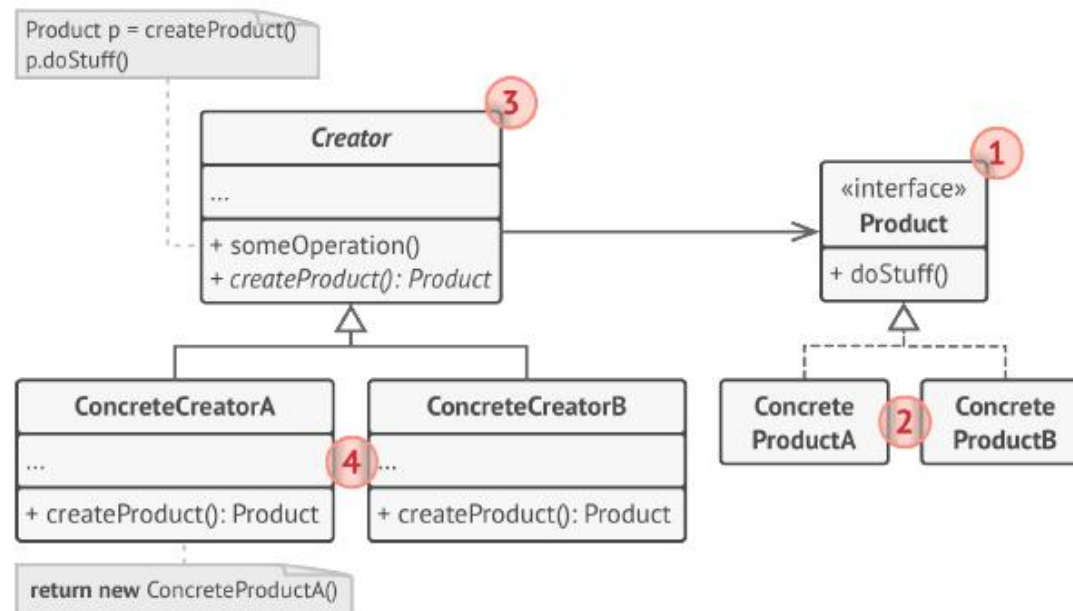
- **The Prototype Pattern** starts with an initialized and instantiated class and copies or clones it to make new instances rather than creating new instances.
- **The Singleton Pattern** is a class of which there can be no more than one instance. It provides a single global point of access to that instance.

The Factory pattern

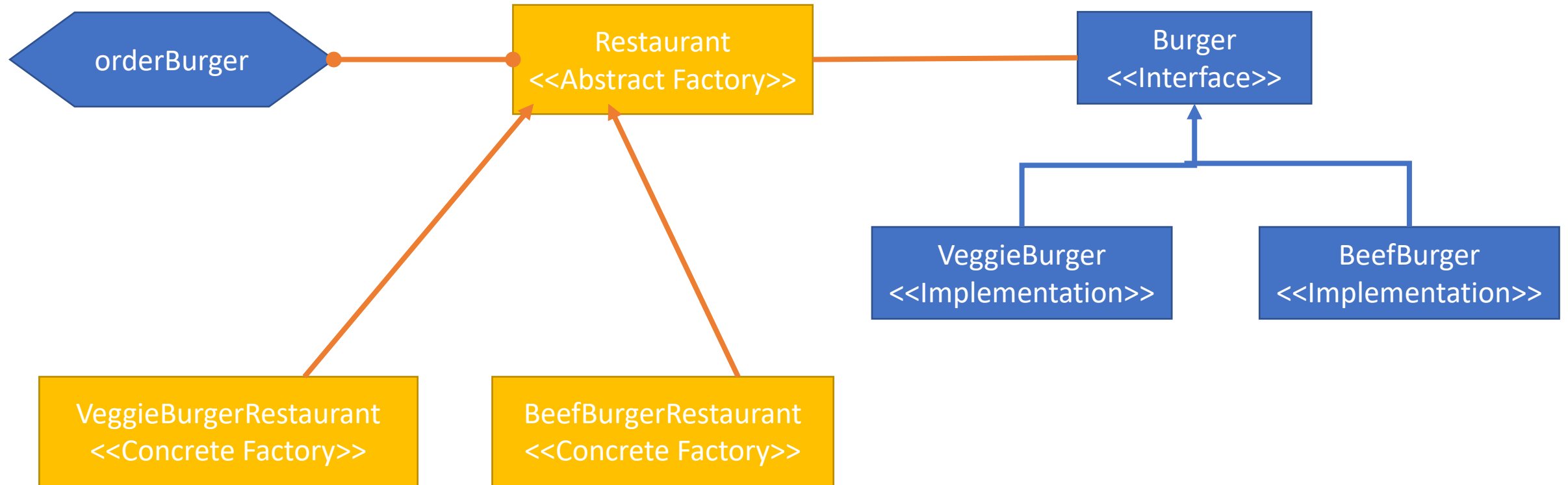
- Gang Of Four Definition
 - Define an interface for creating an object, but let subclasses decide which class to instantiate. The Factory method lets a class defer instantiation it uses to subclasses
- One of the most used design patterns in the real-world applications

Factory Pattern

- This pattern is particularly useful when a class cannot anticipate the class of objects it needs to create, or when a class wants its subclasses to specify the objects it creates.



Factory Example:



```

@GetMapping("/burger/{request}")
public Burger orderBurger(@PathVariable("request") String request){
    if("BEEF".equals(request)){
        Restaurant beefBurgerRestaurant = new BeefBurgerRestaurant();
        return beefBurgerRestaurant.orderBurger();
    } else{
        Restaurant veggieBurgerRestaurant = new VeggieBurgerRestaurant();
        return veggieBurgerRestaurant.orderBurger();
    }
}

```

```

public class BeefBurgerRestaurant extends Restaurant{

    @Override
    public Burger createBurger() {
        return new BeefBurger();
    }

}

```

```

public abstract class Restaurant {
    public Burger orderBurger(){
        Burger burger = createBurger();
        burger.prepare();

        return burger;
    }

    public abstract Burger createBurger();
}

```

Exercise 1

- Extend the Burger Restaurant
 - Base on the factory example, extend the code following Factory Pattern to accept the ordering of Chicken Burger.
- Do it now!

Exercise 2

- A document management system in which documents can be in different formats like Word, PDF, Excel, etc. The documents need to be opened, saved, and closed. How Factory Pattern handle this diversity, keeping in mind that in the future we may have to introduce more document types?

Observation

- In standard programming, a "High-Level" class (like a DocumentManager) often creates an instance of a "Low-Level" class (like a PDFDocument) directly using the new keyword.
- **The Issue:** The High-Level class is now "stuck" to that specific Low-Level class. If you want to switch to an ExcelDocument, you have to modify the code inside DocumentManager.
- This violates the Dependency Inversion Principle because a high-level module depends on a concrete implementation.

Observation – The Solution

- Dependency Inversion suggests two things:
 - High-level modules should not depend on low-level modules. Both should depend on **abstractions** (interfaces).
 - Abstractions should not depend on details. Details should depend on abstractions.
- By introducing an interface (e.g., IDocument), the DocumentManager no longer cares *which* writer it's using, as long as it follows the rules of the interface.

Observation – Why Factory Pattern?

- Even if you use an interface, you still face a practical problem:
 - **Somewhere, someone has to call new PDFDocument().**
 - If you do that inside your High-Level class, you've re-introduced the coupling you just tried to avoid.
- Instead of the High-Level class instantiating the object itself, it calls a "Factory."
- The Factory Method handles the "dirty work" of creating the specific object and returns it as the abstract interface.

Observation – Why Factory Pattern?

- Imagine you have 50 different classes in your app that need to process documents.
- Without a Factory: All 50 classes have `new PDFDocument()`. If you switch to `ExcelDocument()`, you must find and change 50 files.
- With a Factory: All 50 classes call `factory.openDocument()`. Only one file (the Factory) contains the *new* keyword.

Observation – Factory Pattern with DI

- In a well-designed system, your 50 classes don't *create* the factory; they **receive** it.
- Imagine a UserDashboard class. Instead of it reaching out to create a factory, you "inject" the factory through the constructor.

```
class UserDashboard {  
    private DocumentFactory documentFactory;  
  
    // The dashboard doesn't care WHERE this factory came from  
    public UserDashboard(DocumentFactory documentFactory) {  
        this.documentFactory = documentFactory;  
    }  
  
    public void onClick() {  
        documentFactory.openDocument().save();  
    }  
}
```

Observation – Factory Pattern with Composition Root

- Where does the *new PDFDocumentFactory()* finally happen? **Only once**, at the very start of your program.
- By using the Factory Method, you have built a **separation** between your "Business Logic" and your "Tooling."

```
class DocumentManagementSystem {  
    Run | Debug  
    public static void main(String[] args) {  
        DocumentFactory myFactory = new WordDocumentFactory();  
  
        UserDashboard dashboard = new UserDashboard(myFactory);  
        DocumentPrinting printing = new DocumentPrinting(myFactory);  
        OrderProcessor orders = new OrderProcessor(myFactory);  
        // ... and so on  
    }  
}
```

Builder Pattern

- The Builder pattern is a creational pattern –it's used to create and configure complex objects.
- Use to simplify the complexity of the object initialization by letting you construct an object step by step.

Builder Pattern – The Problem to Solve

Car

- id : int
- brand : String
- model : String
- color : String
- nbrDoors : int
- screenType : String
- weight : double
- height : double

```
Car car = new Car(id, brand, model, color, nbrDoors,  
                 screenType, weight, height);
```

```
Car car = new Car(id, brand, null, color, nbrDoors,  
                 null, weight, null);
```

```
new Car(id, brand, model);  
new Car(id, screenType, weight, height);  
new Car(id, brand, model, color, nbrDoors);  
new Car(id, brand, screenType, weight, height);  
.  
.  
.
```

Builder Pattern

- The builder pattern extract the object construction code out of its own class and move it to a separate objects called Builder.

```
public class CarBuilder {  
    private int id;  
    private String brand;  
    private String model;  
    private String color;  
    ...  
  
    public CarBuilder id(int id) {  
        this.id = id;  
    }  
  
    public CarBuilder brand(String brand) {  
        this.brand = brand;  
    }  
  
    public CarBuilder model(String model) {  
        this.model = model;  
    }  
  
    public CarBuilder color(String color) {  
        this.color = color;  
    }  
    ...  
  
    public Car build() {  
        return new Car(id, brand, model, color);  
    }  
}
```

```
public class Car {  
    private final int id;  
    private final String brand;  
    private final String model;  
    private final String color;  
    ...  
  
    Car(int id, String brand, String model,  
        String color, ...) {  
  
        this.id = id;  
        this.brand = brand;  
        this.model = model;  
        this.color = color;  
        ...  
    }  
}
```

Using the Builder Class

```
Car car = new CarBuilder()  
    .id(2122)  
    .brand("Bugatti")  
    .model("Chiron")  
    .color("Blue")  
    .nbrDoors(2)  
    .engine("8L")  
    .height(115)  
    .build();
```

```
Car car = new CarBuilder()  
    .id(2123)  
    .brand("Bugatti")  
    .model("Divo")  
    .color("Blue")  
    .nbrDoors(2)  
    .engine("8L")  
    .height(121)  
    .build();
```

```
Car car = new CarBuilder()  
    .id(2124)  
    .brand("Bugatti")  
    .model("Centodieci")  
    .color("Blue")  
    .nbrDoors(2)  
    .engine("8L")  
    .height(121)  
    .build();
```

```
Car car = new CarBuilder()  
    .id(1122)  
    .brand("Lamborghini")  
    .model("Aventador")  
    .color("Yellow")  
    .nbrDoors(2)  
    .engine("6.5L")  
    .height(115)  
    .build();
```

```
Car car = new CarBuilder()  
    .id(1123)  
    .brand("Lamborghini")  
    .model("Huracán")  
    .color("Yellow")  
    .nbrDoors(2)  
    .engine("5.2L")  
    .height(115)  
    .build();
```

```
Car car = new CarBuilder()  
    .id(1122)  
    .brand("Lamborghini")  
    .model("Aventador")  
    .color("Yellow")  
    .nbrDoors(2)  
    .engine("6.5L")  
    .height(115)  
    .build();
```

Enhancement To The Builder

- Referring to the previous slide, can we make the construction of the object easier?
- Introduce a Director class to manage the order in which we should call the construction steps so that we can reuse specific configurations of the products we are building.
- Director is an optional class in the pattern.
- It helps to hide the details of the product construction from the client code.

Director

```
public class Director {  
  
    public void buildBugatti(CarBuilder builder) {  
        builder.brand("Bugatti")  
            .color("Blue")  
            .nbrDoors(2)  
            .engine("8L")  
            .height(115);  
    }  
  
    public void buildLambo(CarBuilder builder) {  
        builder.brand("Lamborghini")  
            .model("Aventador")  
            .color("Yellow")  
            .nbrDoors(2)  
            .height(115);  
    }  
  
}
```

```
Director director = new Director();  
CarBuilder builder = new CarBuilder();  
director.buildBugatti(builder);  
Car car = builder.build();
```

Application of Builder Pattern

- Use the Builder pattern to get rid of a constructor with many optional parameters.
- Use the Builder pattern when you want your code to be able to create different representations of same product without confusing the client.

Scenario Where We Use Builder Pattern

- A Robot might have many parts like head, body, arms, legs, etc., and each of these parts might have many options. For example, the robot's body could be made of metal, plastic, or some advanced alloy. The robot's head might contain different types of sensors and processors.
- When you're dealing with an object that has a complex configuration. For instance, consider an online pizza ordering system. Pizzas can have a wide variety of options (e.g., crust type, size, toppings, cheese, sauce) and the ordering process can be effectively managed using a Builder pattern.
- Consider a scenario where you are designing a complex 3D game with different types of characters. Each character in the game could have multiple attributes like strength, speed, health, weapon, and abilities. Building these characters directly could be quite complex as each character could have different kinds of abilities, weapons and attributes.

Structural Patterns

Structural design patterns are essential because they provide a manner to simplify and manage the relationships between different software entities. They guide us in composing objects to form larger structures while keeping these structures flexible and efficient.

- **Code Reusability:** They provide solutions to problems that occur frequently in software design, and by using these patterns, we can reuse solutions that have been proven effective.
- **Improved Modularity:** Structural patterns can greatly improve modularity in a program by providing ways to decouple systems and create interfaces.
- **Code Maintenance:** They make the code easier to understand and maintain by ensuring that classes and objects are well-organized and well-structured.
- **Flexibility:** They enable us to change the system's components independently, which leads to higher flexibility.

Structural Patterns

- **Adapter Pattern** acts as a connector between two incompatible interfaces that otherwise cannot be connected directly. An adapter wraps an existing class with a new interface so that it becomes compatible with the client's interface.
- **Bridge Pattern** is to decouple an abstraction from its implementation so that the two can vary independently.
- **Composite Pattern** is meant to allow treating individual objects and compositions of objects, or “composites” in the same way.
- **Decorator Pattern** can be used to attach additional responsibilities to an object either statically or dynamically.

Structural Patterns

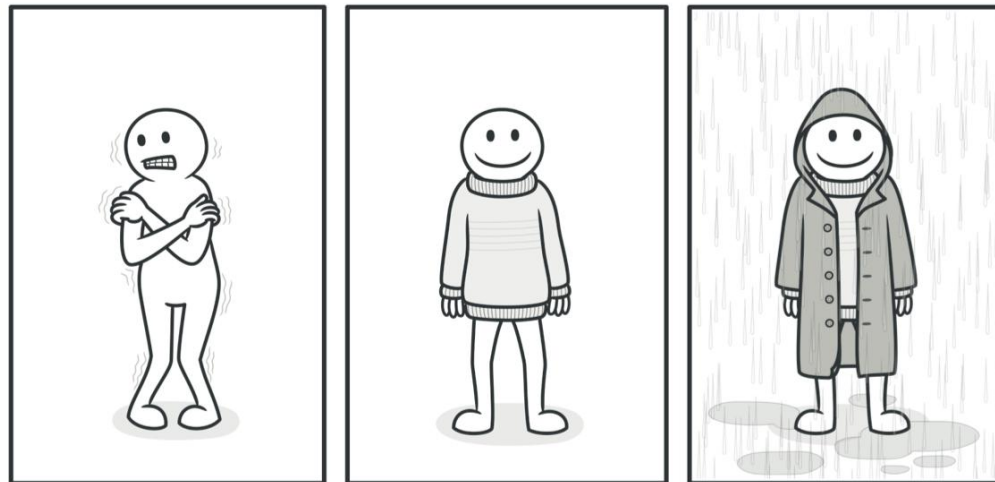
- **Facade Pattern** encapsulates a complex subsystem behind a simple interface. It hides much of the complexity and makes the subsystem easy to use.
- **Flyweight Pattern** allows programs to support vast quantities of objects by keeping their memory consumption low. Pattern achieves it by sharing parts of object state between multiple objects.
- **Proxy Pattern** specifies a design where substitute or placeholder object is put in-place of the actual target object to control access to it. Client accesses the proxy object to work with the target object.

Decorator Pattern

- The **decorator design pattern** allows us to dynamically add functionality and behavior to an object without affecting the behavior of other existing objects in the same class.
- We use inheritance to extend the behavior of the class. This takes place at compile time, and all the instances of that class get the extended behavior.
- Apply decorator to an individual object based on our requirement and choice.

Decorator Pattern

- Decorator a.k.a. Wrapper
- Uses abstract classes or interfaces with the composition to implement the wrapper.
- Decorator pattern create decorator classes, which wrap the original class and provide additional functionality by keeping the class methods' signature unchanged.

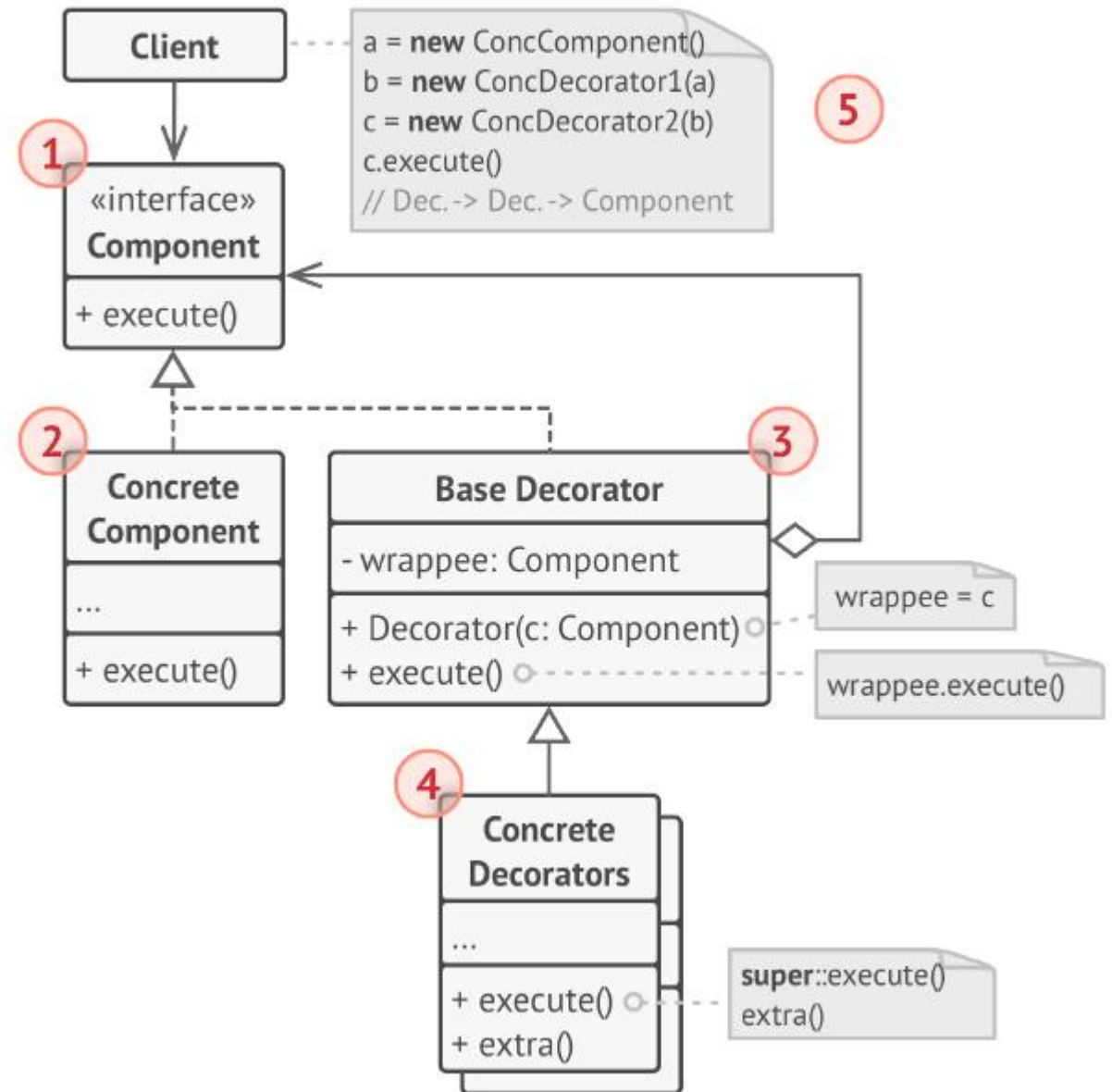


You get a combined effect from wearing multiple pieces of clothing.

Decorator Pattern

– The Structure

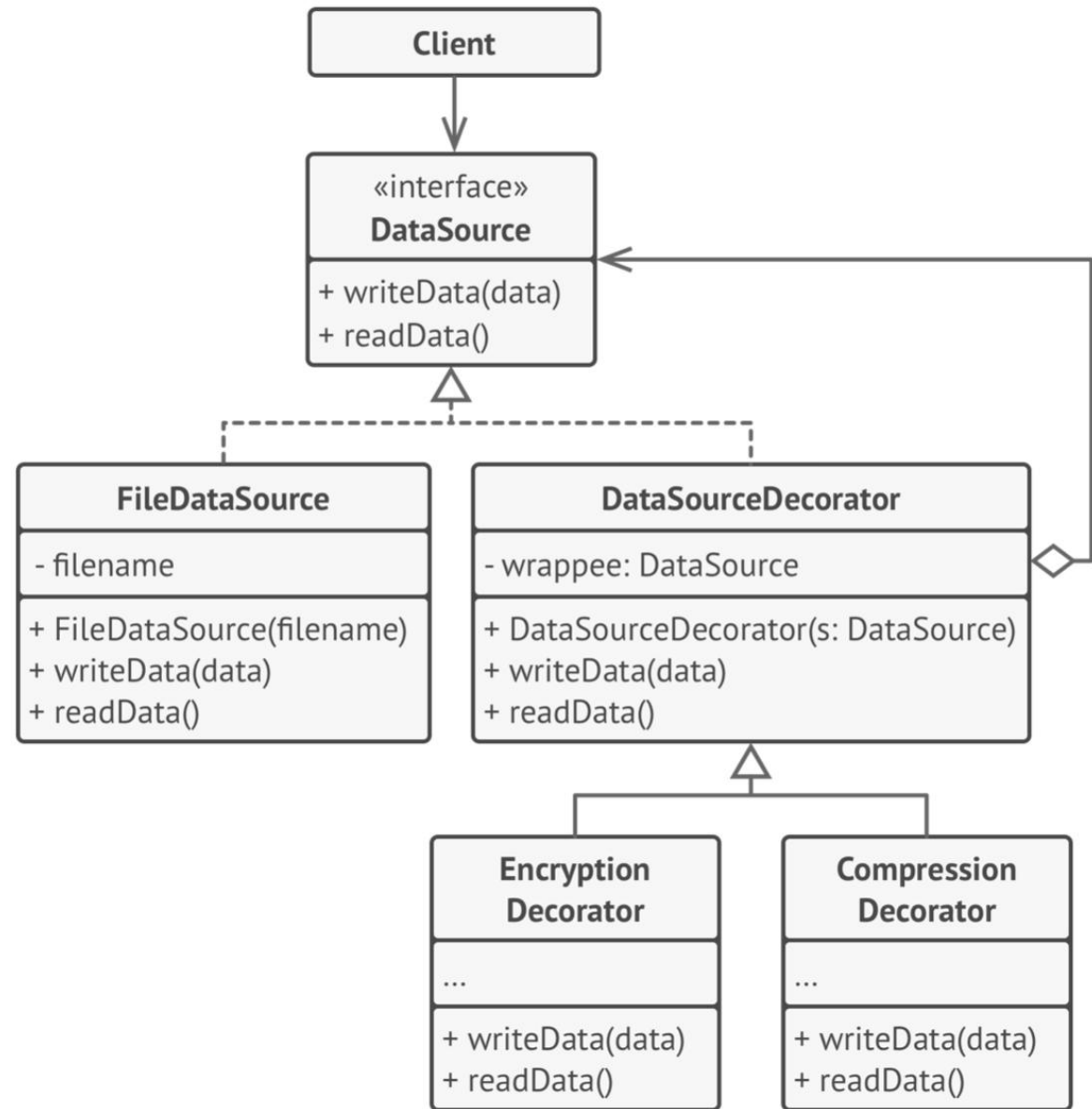
- Also check out the sample Java code, Robot.



Key Components

- The ***Component*** declares the common interface for both wrappers and wrapped objects.
- ***Concrete Component*** is a class of objects being wrapped. It defines the basic behavior, which can be altered by decorators.
- The ***Base Decorator*** class has a field for referencing a wrapped object. The field's type should be declared as the component interface so it can contain both concrete components and decorators. The base decorator delegates all operations to the wrapped object.
- ***Concrete Decorator*** define extra behaviors that can be added to components dynamically. Concrete decorators override methods of the base decorator and execute their behavior either before or after calling the parent method.

Decorator Pattern – Data Source Example



The encryption and compression decorators example.

Example

- Suppose we have a Robot interface, which included Move() and Cook(). You want some of your robot instances to do more things.

```
1 public interface Robot{  
2     public void Move(int x, int y, int speed);  
3  
4     public void Cook();  
5 }
```

```
1 class JapaneseRobot implements Robot{  
2  
3     @Override  
4     public void Move(int x, int y, int speed) {  
5         // TODO Auto-generated method stub  
6         System.out.println("Japanese Robot moved to " + x + "  
7     }  
8  
9     @Override  
10    public void Cook() {  
11        // TODO Auto-generated method stub  
12        System.out.println("Cooking Japanese food");  
13    }  
14 }
```

```
class PlayRobot {  
    private List<Robot> robots = new ArrayList<Robot>();  
  
    public void AddRobot(Robot r){  
        robots.add(r);  
    }  
  
    public void AllRobotsCook(){  
        robots.stream().forEach(r -> r.Cook());  
    }  
}
```


Example

- I want my third robot to cook according to customer's gender. Female get a small portion, male get a big portion.

```
15 public static void main(String[] args){
16     PlayRobot play = new PlayRobot();
17
18     Robot r = new ChineseRobot();
19     play.AddRobot(r);
20
21     r = new JapaneseRobot();
22     play.AddRobot(r);
23
24     r = new ChineseRobot();
25     play.AddRobot(r);
26
27     play.AllRobotsCook();
28 }
```

PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL

```
PS S:\我的资料库\My Library\xjtlu\Modules\CSE306 - Software Engineering 2\Code Demo\Patterns\Decorator P
ring 2\Code Demo\Patterns\Decorator Pattern\Robot'; & 'C:\Users\ra\vscode\extensions\vscjava.vscod
n\java.exe' '-Dfile.encoding=UTF-8' '-cp' 'C:\Users\ra\AppData\Roaming\Code\User\workspaceStorage\10b5bb
t'
Cooking Chinese food
Cooking Japanese food
Cooking Chinese food
PS S:\我的资料库\My Library\xjtlu\Modules\CSE306 - Software Engineering 2\Code Demo\Patterns\Decorator P
```

Example

- A decorator implements Robot interface and hold an instance of Robot (r), which would like to have additional functionalities.
- The decorator must ensure all methods in the interface are implemented by delegating to robot r.

```
1 public class RobotDecorator implements Robot{
2     private Robot r;
3
4     protected RobotDecorator(Robot r){
5         this.r = r;
6     }
7
8     @Override
9     public void Move(int x, int y, int speed) {
10         r.Move(x, y, speed);
11     }
12
13     @Override
14     public void Cook() {
15         r.Cook();
16     }
17 }
```

Example

- To add functionalities, the class (RationalRobotDecorator) extends RobotDecorator and adding new functions by overriding the decorator's methods.

```
1 public class RationalRobotDecorator extends RobotDecorator{
2     private boolean gender;
3
4     public RationalRobotDecorator(Robot r, boolean gender) {
5         super(r);
6         this.gender = gender; // True represent FEMALE
7     }
8
9     @Override
10    public void Move(int x, int y, int speed) {
11        super.Move(x, y, speed);
12    }
13
14    @Override
15    public void Cook() {
16        if(gender){
17            System.out.println("I will cook you a small portion");
18        } else {
19            System.out.println("I will cook you a big portion");
20        }
21        super.Cook();
22    }
23 }
```

Application of Decorator Pattern

- Use the Decorator pattern when you need to be able to assign extra behaviors to objects at runtime without breaking the code that uses these objects.
- Use the pattern when it's awkward or not possible to extend (class with final keyword) an object's behavior using inheritance.

Stream I/O (The "Classic" Industry Implementation)

- If you have ever written code in Java or C#, you have used the Decorator Pattern without realizing it. The entire standard library for Input/Output (I/O) is built this way.
- **The Implementation:**
 - You start with a basic *FileInputStream* (reads raw bytes).
 - You wrap it in a *BufferedInputStream* (adds the "behavior" of buffering for speed).
 - You wrap that in a *DataInputStream* (adds the "behavior" of reading specific data types like Integers or Strings).

```
InputStream file = new FileInputStream("data.txt");  
InputStream buffered = new BufferedInputStream(file);  
DataInputStream data = new DataInputStream(buffered);
```

Exercise 3

- A new type of Robot is invented. It is smarter because it can apply deep learning to cook better.
- Now, any Robot can be a rational robot at the same time smart robot.
- Enhance the above example by implement a SmartRobotDecorator.
- Illustrate that a Robot can be rational and smart at the same time.

Exercise 4

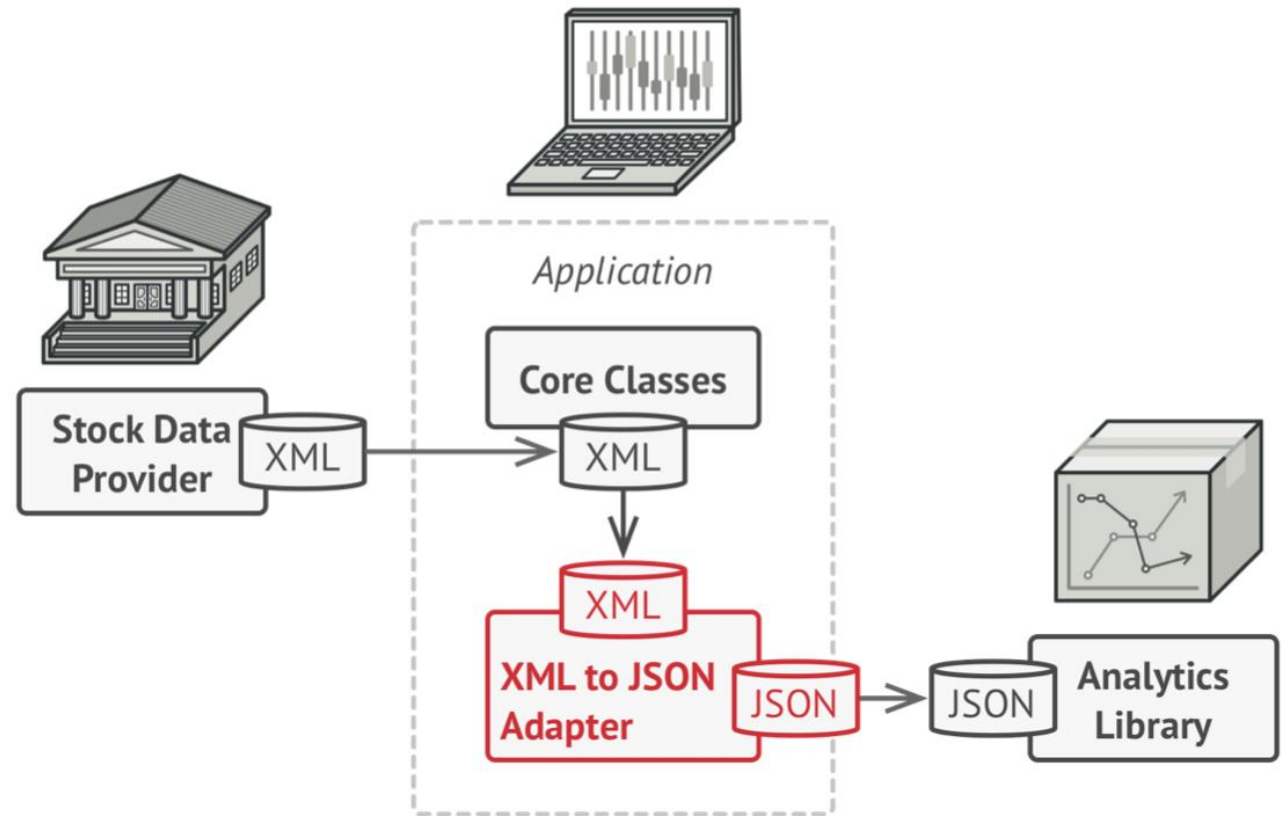
- Let's think about a cafe scenario. When you go to a cafe, you usually start with a basic type of coffee, then you customize it by adding extra features like milk, sugar, flavors, whipped cream, etc. Plus, each of these additions affects the final cost of the coffee.
- Implementing this in a straightforward way could quickly become complicated and messy, especially if you want to be able to add, remove, and modify these customizations dynamically.
- This is a perfect scenario for the Decorator pattern. In this pattern, a class represents the base object (in this case, a plain coffee), while other decorator classes represent the optional extras. The decorators wrap the base class and other decorators, forming a chain of objects where each object adds its own behavior (and cost) to the final product.
- Develop the solution

Adapter Pattern

- The adapter pattern makes two incompatible interfaces compatible without changing their existing code.
- Adapter patterns use a single class (the adapter class) to join functionalities of independent or incompatible interfaces/classes.
- The adapter pattern also is known as the wrapper, an alternative naming shared with the decorator design pattern.
- The adapter implements the interface of one object and wraps the other one.

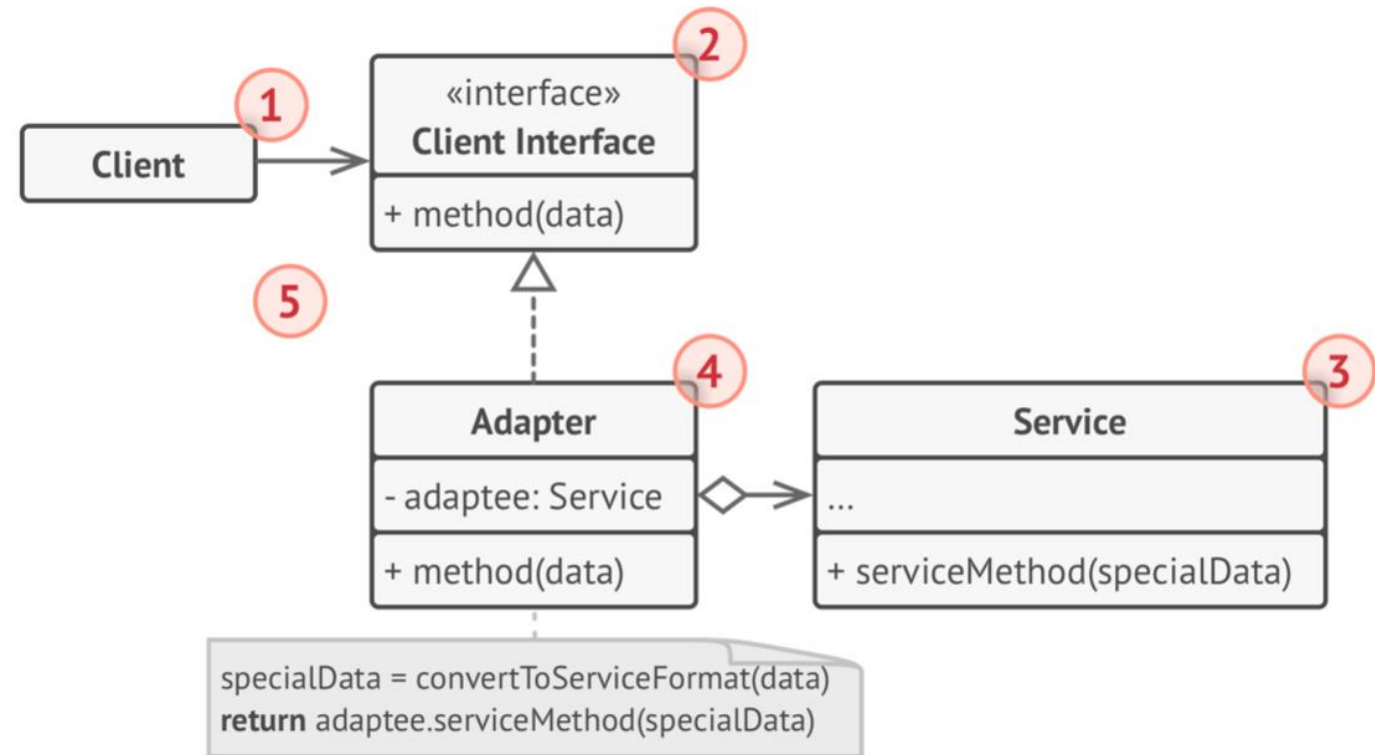
Adapter Pattern

- The adapter gets an interface, compatible with one of the existing objects.
- Using this interface, the existing object can safely call the adapter's methods.
- Upon receiving a call, the adapter passes the request to the second object, but in a format and order that the second object expects.



Adapter Pattern – The Structure

- Also check out the sample Java code, Weapon.



Key Components

- The **Client Interface** describes a protocol that other classes must follow to be able to collaborate with the client code.
- The **Service** is some useful class (usually 3rd-party or legacy). The client can't use this class directly because it has an incompatible interface.
- The **Adapter** is a class that's able to work with both the client and the service: it implements the client interface, while wrapping the service object. The adapter receives calls from the client via the adapter interface and translates them into calls to the wrapped service object in a format it can understand.

Example

- Suppose we have a project of controlling robots, in which we are required to develop different kinds of robots that will be used in the ***PlayRobot*** via a common interface called ***Robot***.

```
1 interface Robot{  
2     public void Move(int x, int y, int speed);  
3  
4     public void Cook();  
5 }
```

```
class PlayRobot {  
    private List<Robot> robots = new ArrayList<Robot>();  
  
    public void AddRobot(Robot r){  
        robots.add(r);  
    }  
  
    public void AllRobotsCook(){  
        robots.stream().forEach(r -> r.Cook());  
    }  
}
```

Example

- As we progress, we come to know that there are some extra robots that are already developed either by some other team within our organization. Or, we have a third-party API, which is available to us.

```
public abstract class Machine{  
    private int speed;  
  
    public void SetSpeed(int speed){  
        this.speed = speed;  
    }  
  
    public int GetSpeed(){  
        return speed;  
    }  
  
    public abstract void Kill();  
  
    public abstract void Run(int x, int y);  
}
```

```
public class MilitaryMachine extends Machine{  
    @Override  
    public void Kill() {  
        // TODO Auto-generated method stub  
        System.out.println("Aim for the heart");  
    }  
  
    @Override  
    public void Run(int x, int y) {  
        // TODO Auto-generated method stub  
        System.out.println("Soldier runs to " + x + "," + y + " at the speed of " + GetSpeed());  
    }  
}
```

```
public class ZombieMachine extends Machine{  
    @Override  
    public void Kill() {  
        // TODO Auto-generated method stub  
        System.out.println("Aim for the brain");  
    }  
  
    @Override  
    public void Run(int x, int y) {
```

Applying adapter pattern

```
class MachineAdapter implements Robot{
    private Machine m;

    public MachineAdapter(Machine m){
        this.m = m;
    }

    @Override
    public void Move(int x, int y, int speed) {
        // TODO Auto-generated method stub
        m.SetSpeed(speed);
        m.Run(x, y);
    }

    @Override
    public void Cook() {
        // TODO Auto-generated method stub
        if(m instanceof ZombieMachine){
            System.out.println("Cooking Zombie food");
        }else if(m instanceof MilitaryMachine){
            System.out.println("Cooking Soldier food");
        }else{
            System.out.println("Cooking cannot be completed");
        }
    }
}
```

Application of Adapter Pattern

- Use the Adapter class when you want to use some existing class, but its interface isn't compatible with the rest of your code.
- Use the pattern when you want to reuse several existing sub-classes that lack some common functionality that can't be added to the superclass.

Exercise 5

- Consider an e-commerce scenario where users can make payments using different payment systems like PayPal, CreditCards, and cryptocurrencies.
- Each of these payment systems has a different API and integration process. When a user chooses a payment method, your system should process the payment using the respective payment system's API. Directly integrating these different APIs into your system can lead to a lot of complexities and code clutter.
- Show and discuss your implementation.

The End